Project

Bookoa.co.kr

open API 온라인 서점

임승찬 https://mynarne.github.io/Bookstore_Project/

Table

1 Timeline

2 기획 과정 (서점 선택)
페르소나

3 UI
Main Page
Sub Page

4 Review

Timelines

Timelines

03.16

기획
(서점 사이트 선택)

03.17 ~ 03.18

UI 기획

03.18 ~ 03.27

Main Page,
Sub Page 구현

03.30 ~ 03.31

검토, 문서 작업

기획

서점 선정

북코아

서점 선택 : 북코아

처음 접속 했을 때 중고 서점이라는 느낌이 없고,
복잡한 광고 없이 메인 페이지에 책만 보여주는 간결함 때문에 선택

Persona

기획

페르소나

성명 : 김민준 (32세, 남성)

직업 : 대기업 5년 차 영업직

좌우명 : 중고 가격으로 새 지식을 얻는다

라이프 스타일

- 실용주의적 가성비 컬렉터
- 절판본, 디자인 서적 수집이 취미

핵심 니즈

1. 상태의 투명성

중고 특성 상 낙서 또는 파손에 민감하므로 신뢰할 수 있는 검수 시스템, 고화질 사진 확인을 원함

2. 배송비 최적화

여러 판매자에게 구매하더라도 배송비를 한 번만 내고 싶은 '합배송' 기능 갈망

3. 간편한 판매 절차

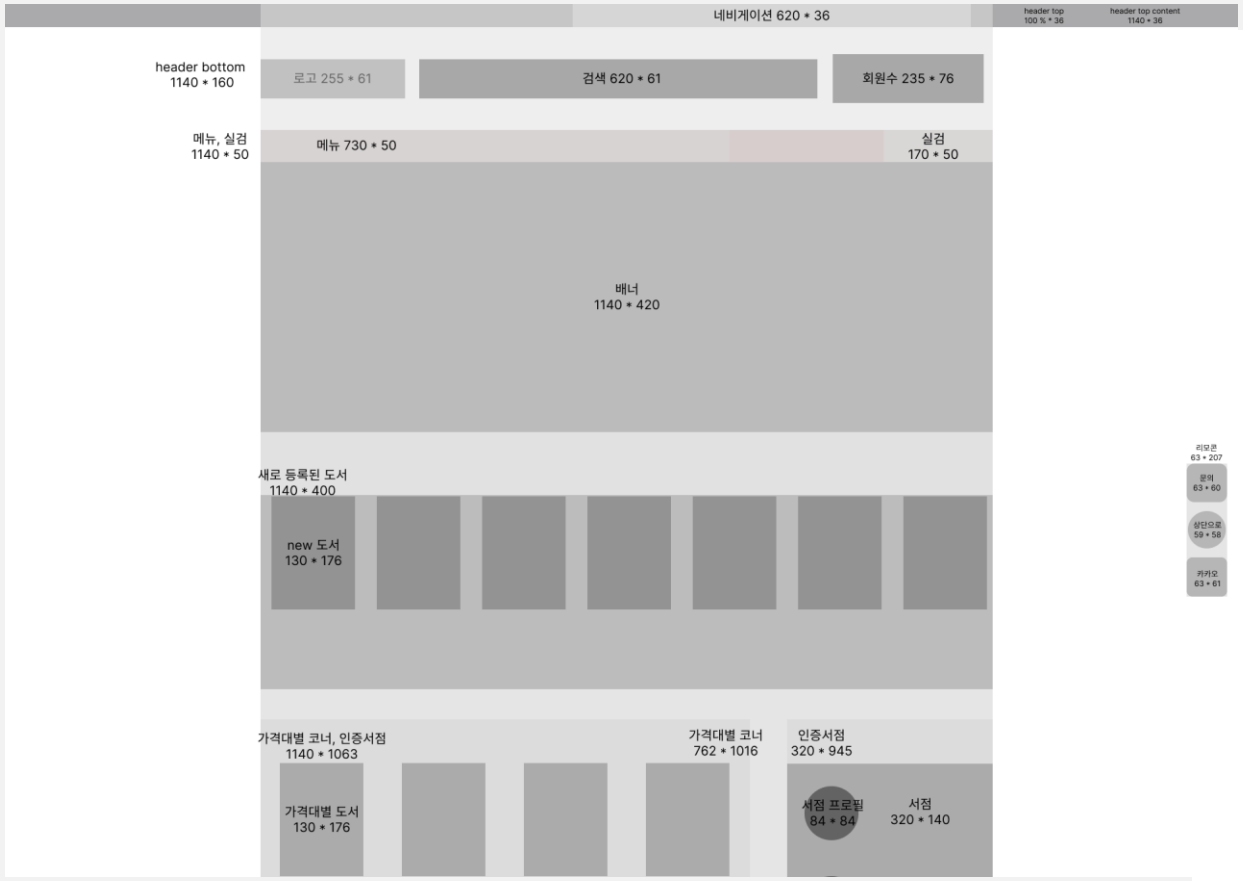
소장 도서를 판매하는 경우에도 '원클릭 방문 매입' 서비스 선호

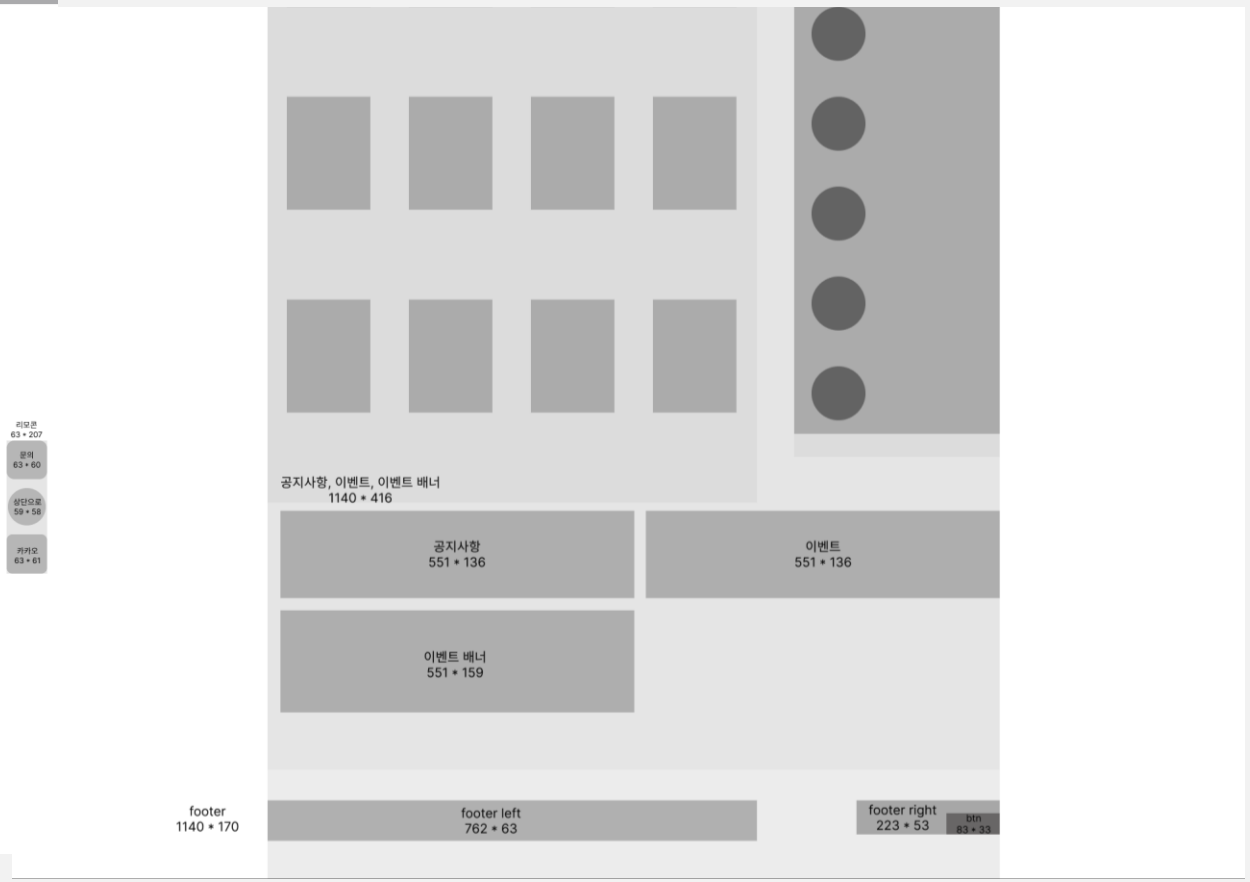
페인 포인트

'최상' 등급의 책을 구매했으나 실제 상태가 기대 이하일 경우 플랫폼 신뢰도 급락

WireFrame

UI 기획





서점 사이트 와이어프레임 작업
각 요소 별 사이즈 측정 후 기재

Main Page

메인 페이지

CSS

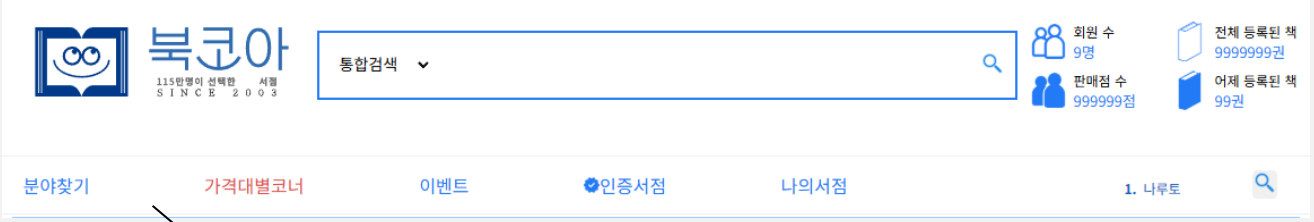
```css
.sub_genre, .sub_price, .sub_event {
  display: none !important;
  position: absolute;
  top: 100%;
  left: 0;
  background-color: #fff;
  border: 1px solid #ddd;
  box-shadow: 0 4px 10px rgba(0,0,0,0.1);
  padding: 20px;
  z-index: 100;
  list-style: none;
}
// 헤더 리스트
```

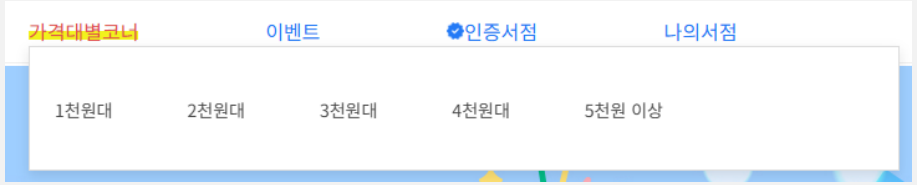
평소에는 display: none 부여하여 감춤

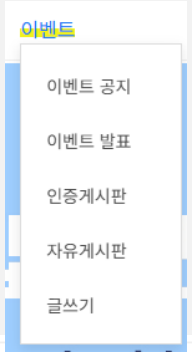
JS

```js
document.addEventListener('DOMContentLoaded', () => {
  const menus = [
      { parent: '.menu_search_genre', child: '.sub_genre', type: 'grid' },
      { parent: '.menu_price_conner', child: '.sub_price', type: 'flex' },
      { parent: '.menu_event', child: '.sub_event', type: 'block' }
  ];

  menus.forEach(menu => {
      const parentEl = document.querySelector(menu.parent);
      const childEl = document.querySelector(menu.child);

      if (parentEl && childEl) {
          parentEl.addEventListener('mouseenter', () => {
              childEl.style.display = menu.type;
          });
          parentEl.addEventListener('mouseleave', () => {
              childEl.style.display = 'none';
          });
      }
  });
});
```

마우스 포인터가 올라가면
각각 부여한 class에 따라서
화면에 출력한 뒤
메뉴처럼 보이게 함







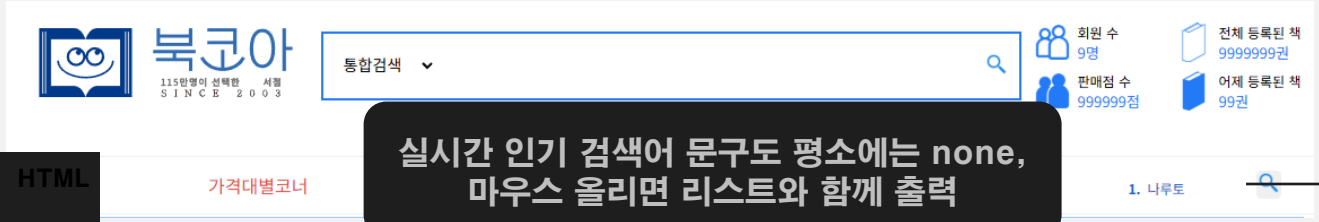
```css
.menu_search_genre:hover .sub_genre {
  display: grid !important;
  grid-template-columns: repeat(4, 1fr);
  gap: 10px 15px;
}
```

CSS

분야찾기 메뉴는
4열로 배치

Main Page

메인 페이지



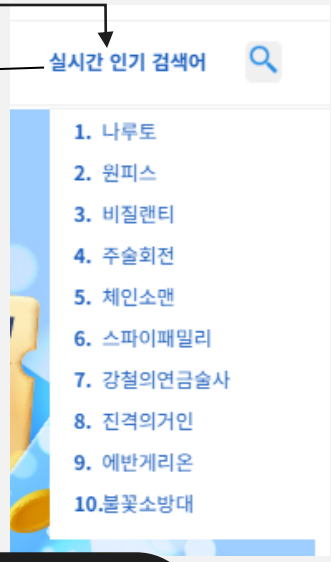
실시간 인기 검색어 문구도 평소에는 none,
마우스 올리면 리스트와 함께 출력

HTML

```html
<!-- 실검 -->
<div class = 'realtimesearch'>
    <p>실시간 인기 검색어</p>
    <div class = 'rts_wrap'>
        <ul class = 'rts_list'>
            <li><span>1.</span>나루토</li>
            <li><span>2.</span>원피스</li>
            <li><span>3.</span>비질랜티</li>
            <li><span>4.</span>주술회전</li>
            <li><span>5.</span>체인소맨</li>
            <li><span>6.</span>스파이패밀리</li>
            <li><span>7.</span>강철의연금술사</li>
            <li><span>8.</span>진격의거인</li>
            <li><span>9.</span>에반게리온</li>
            <li><span>10.</span>불꽃소방대</li>
        </ul>
    </div>
    <button id = "rts_btn" type = "submit"><img id
</div>
```

CSS

```css
.realtimesearch p {
  display: none;
  font-size: 12px;
  font-weight: bold;
  color: #1d60bf;
  margin-top: 5px;
  margin-bottom: 20px;
}

.realtimesearch:hover p {
  display: block;
}
```



CSS

```css
/* 실시간 검색어 */
.realtimesearch {
  width: 170px;
  height: 50px !important;
  display: flex;
  align-items: center;
  justify-content: space-between;
  position: relative;
  box-sizing: border-box;
  overflow: hidden;
  z-index: 10000 !important;
}

.realtimesearch:hover {
  height: 320px;
  flex-direction: column;
  align-items: flex-start;
  padding: 10px;
  overflow: visible;
}
```

CSS

```css
.rts_list {
  width: 200px;
  padding-left: 15px;
  padding-bottom: 10px;
  list-style: none;
  margin: 0;
  position: relative;
  top: 0;
  background-color: white;
}

.realtimesearch:hover .rts_list {
  top: 0;
  transition: none !important;
  border-top: 1px solid #f0f0f0;
}

.rts_list li {
  display: flex;
  align-items: center;
  height: 25px;
  font-size: 12px;
  color: #1d60bf;
}
```

JS

```js
// 실검

document.addEventListener('DOMContentLoaded', () => {
  const rtsList = document.querySelector('.rts_list');
  const container = document.querySelector('.realtimesearch');
  let currentIndex = 0;
  const liHeight = 25;
  const totalItems = rtsList.querySelectorAll('li').length;
  let rollingTimer;

  function startRolling() {
      rollingTimer = setInterval(() => {
          currentIndex++;

          if (currentIndex >= totalItems) {
              currentIndex = 0;
          }

          if (!container.matches(':hover')) {
              rtsList.style.transition = 'top 0.5s ease';
              rtsList.style.top = `-${currentIndex * liHeight}px`;
          }
      }, 2500);
  }

  container.addEventListener('mouseenter', () => {
      clearInterval(rollingTimer);
      rtsList.style.top = '0px';
  });

  container.addEventListener('mouseleave', () => {
      startRolling();
  });

  startRolling();
});
```

setInterval 사용하여
리스트가 일정 시간마다
하나씩 넘어가는 효과 부여

마우스가 올라가면 리스트 전부 출력 및
흘러가는 효과를 잠시 멈춤

Main Page

메인 페이지

```js
async function loadData(genre = '소설') {
  const REST_API_KEY =

  const params = new URLSearchParams({
      query: genre,
      size: 7
  });

  const url = `https://dapi.kakao.com/v3/search/book?${params}`;

  try {
      const response = await fetch(url, {
          method: 'GET',
          headers: {
              Authorization: `KakaoAK ${REST_API_KEY}`
          }
      });

      if (!response.ok) {
          throw new Error(`HTTP 오류 - 상태코드: ${response.status}`);
      }

      const jsonData = await response.json();
      const bookElements1 = document.querySelectorAll('.new_book');


      bookElements1.forEach((book, i) => {
          const bookData1 = jsonData.documents[i];

          if (!bookData1) {
              return;
          }

          book.innerHTML = `
              <a href = 'sub/sub.html?title=${bookData1.title}'>
                  <img class="new_book_img" src="${bookData1.thumbnail}" alt="${bookData1.title} 표지">
                  <div class="new_book_text">
                      <p class="new_book_title">${bookData1.title}</p>
                      <p><b>${bookData1.price.toLocaleString()}원</b></p>
                  </div>
              </a>
          `;
      });

  } catch (error) {
      console.error(`데이터 로딩 중 에러 발생: ${error}`);
  }
}
```

JS

카카오 openAPI를 통해 책 정보 받아옴

.new_book class를 찾아
미리 짜둔 틀에 책 정보 삽입
(이미지, 제목, 가격)

```js
async function loadSubPage(isbn) {
  const response = await fetch('sub/sub.html');
  const htmlText = await response.text();

  document.querySelector('.main_container').innerHTML = htmlText;

  loadData(isbn);
}
```

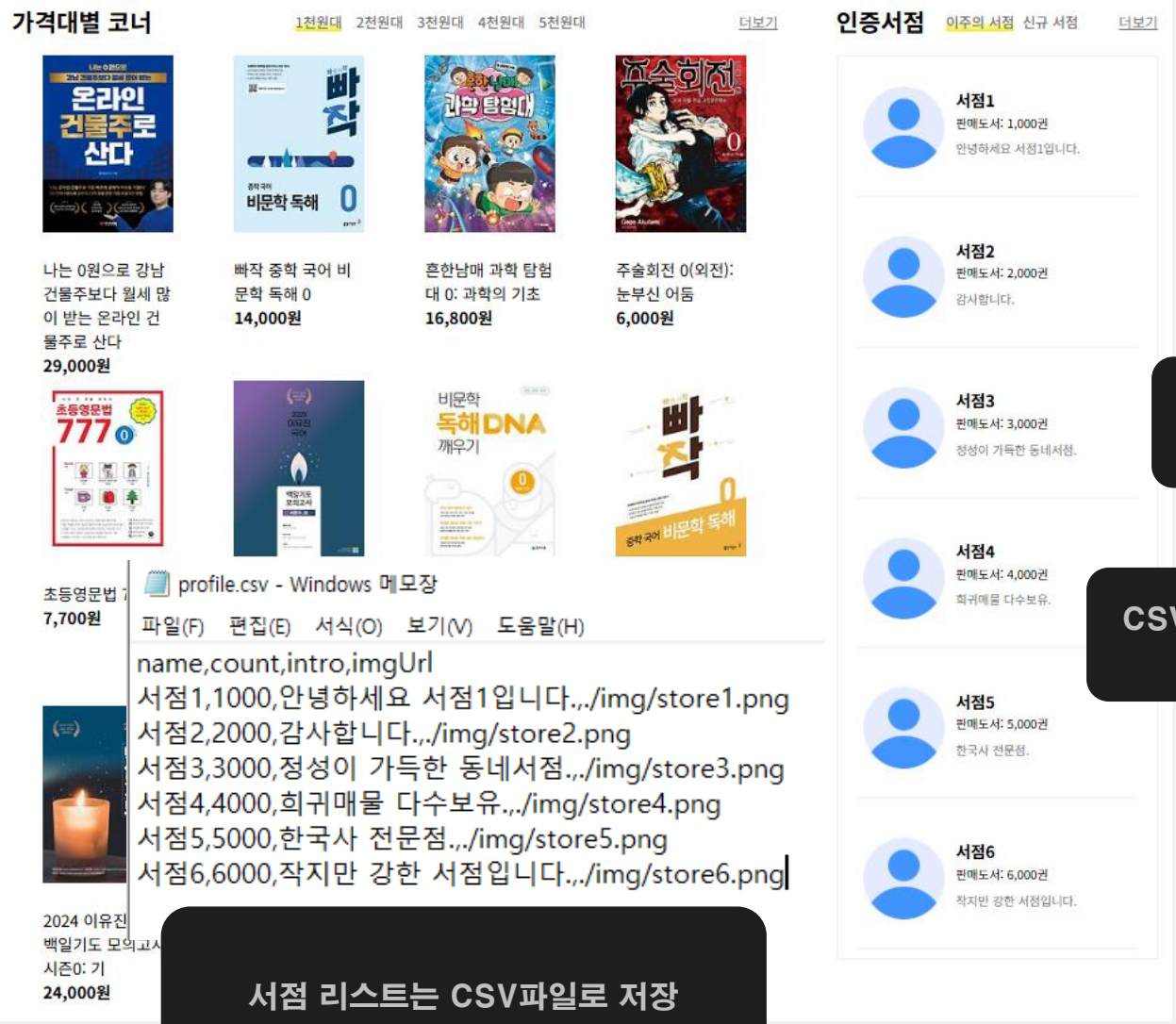
JS

책마다 부여된 ISBN 정보를 받아
서브 페이지로 책 정보를 넘겨줌

하단의 가격대별 코너도 동일한 방식

Main Page

메인 페이지



profile.csv - Windows 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

```
name,count,intro,imgUrl
서점1,1000,안녕하세요 서점1입니다.,./img/store1.png
서점2,2000,감사합니다.,./img/store2.png
서점3,3000,정성이 가득한 동네서점.,./img/store3.png
서점4,4000,희귀매물 다수보유.,./img/store4.png
서점5,5000,한국사 전문점.,./img/store5.png
서점6,6000,작지만 강한 서점입니다.,./img/store6.png
```

서점 리스트는 CSV파일로 저장

HTML

```html
<!-- 인증서점 -->
<div class = 'certi_store_corner'>
    <!-- 인증서점 제목, 메뉴, 더보기-->
    <div class = 'certi_store_menu'>
        <h2>인증서점</h2>
        <nav class = 'csc_nav'>
            <ul>
                <li><a href = "#" id = "bookoa_gray">이주의 서점</a></li>
                <li><a href = "#" id = "bookoa_gray">신규 서점</a></li>
            </ul>
        </nav>
        <a href = "#" class = "more_view" id="bookoa_gray">더보기</a>
    </div>
    <div class = 'certi_store_content' id = 'store_list'>
        <!-- 서점 리스트 여기 들어감 -->
    </div>
```

저장해둔 CSV 파일을 JS를 통해 정보 불러옴

CSV에서 ,로 데이터를 분리 한 뒤 ,을 기준으로 문자열 나눔

JS

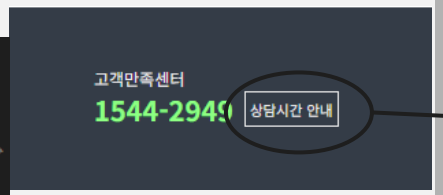
```js
async function loadStoreProfiles() {
  try {
    const response = await fetch('./javascript/profile.csv');
    const data = await response.text();

    const rows = data.split('\n').slice(1);
    const storeList = document.getElementById('store_list');
    storeList.innerHTML = '';

    rows.forEach(row => {
      const cols = row.split(',');
      if (cols.length < 3) return;

      const [name, count, intro, imgUrl] = cols;

      const storeHTML = `
        <div class='certi_store'>
          <div class='certi_store_profile'>
            <img src="${imgUrl.trim() || './img/icon.png'}" alt="${name}">
          </div>
          <div class='certi_store_txt'>
            <h4>${name}</h4>
            <p>판매도서: ${Number(count).toLocaleString()}권</p>
            <p class='introduce'>${intro}</p>
          </div>
        </div>
      `;
      storeList.insertAdjacentHTML('beforeend', storeHTML);
    });
  } catch (error) {
    console.error("서점 프로필 로딩 실패:", error);
  }
}

// 페이지 로드 시 실행
document.addEventListener('DOMContentLoaded', loadStoreProfiles);
```

Main Page

메인 페이지

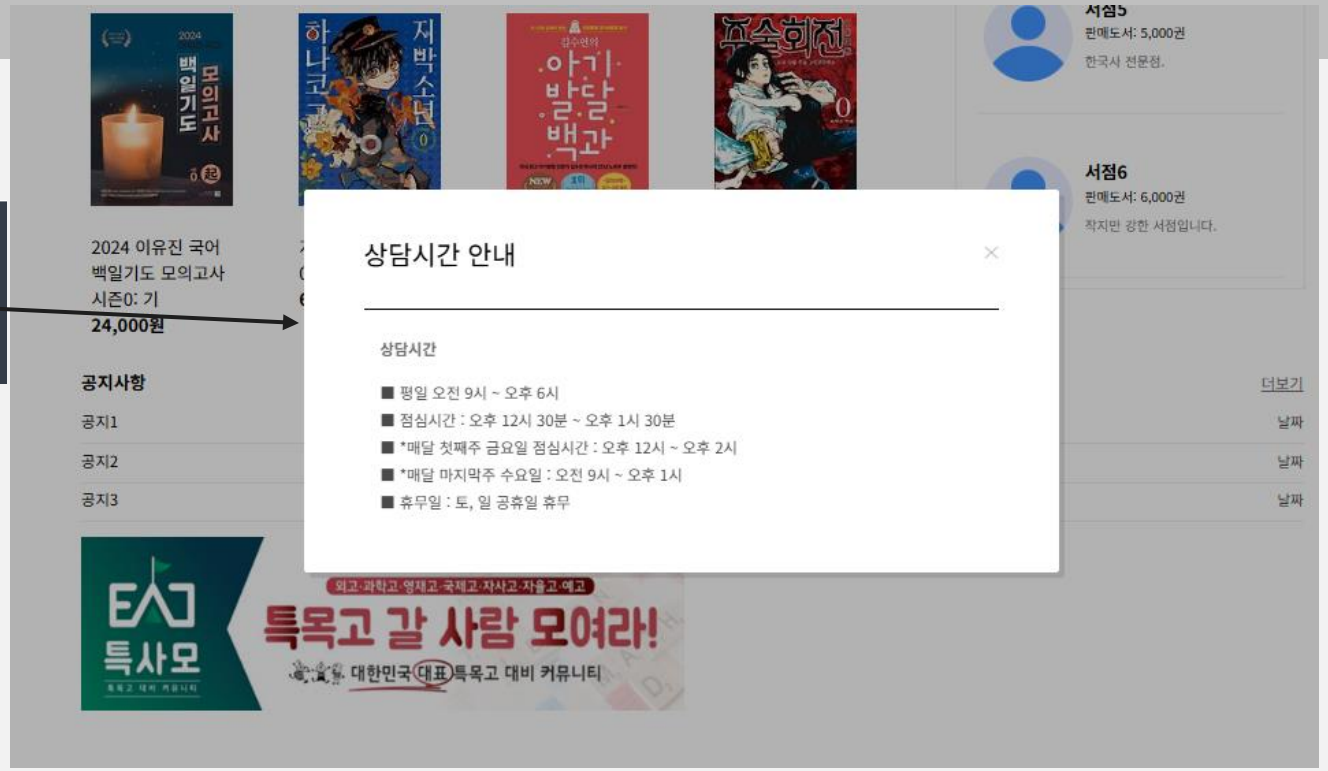
HTML

```html
<div class = 'footer_content_right'>
    <p>고객만족센터</p>
    <div class = 'tel_cuns'>
        <p id ='tel'>1544-2949</p>
        <a href = '#' class = 'cuns_btn'>상담시간 안내</a>
    </div>
</div>
```



HTML

```html
<!-- 상담시간 안내 눌렀을 때 나올 안내 -->
<div class = 'modal_cuns'>
    <div class = 'cuns_notice_main'>
        <div class = 'cuns_notice_title'>
            <p>상담시간 안내</p>
            <button class = 'cuns_notice_close_btn'><img src = './img/close.png' alt = 'icon'></button>
        </div>
        <div class = 'cuns_notice_content'>
            <p><b>상담시간</b></p>
            <ul>
                <li>■ 평일 오전 9시 ~ 오후 6시</li>
                <li>■ 점심시간 : 오후 12시 30분 ~ 오후 1시 30분</li>
                <li>■ *매달 첫째주 금요일 점심시간 : 오후 12시 ~ 오후 2시</li>
                <li>■ *매달 마지막주 수요일 : 오전 9시 ~ 오후 1시</li>
                <li>■ 휴무일 : 토, 일 공휴일 휴무</li>
            </ul>
        </div>
    </div>
</div>
```



CSS

```css
.modal_cuns {
  display: none;
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  z-index: 9999;
  justify-content: center;
  align-items: center;
}
```

CSS

```css
.cuns_notice_main {
  width: 690px;
  height: 350px;
  margin: 0 auto;
  background-color: ■white;
  border-radius: 3px;
  box-shadow: 5px 5px □rgba(0,0,0,0.1);
}
```

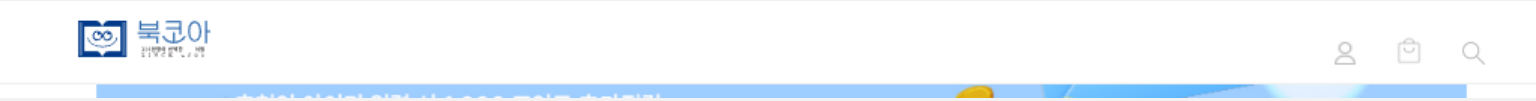
JS

```js
// modal_cuns

const modal = document.querySelector('.modal_cuns');
const openBtn = document.querySelector('.cuns_btn');
const closeBtn = document.querySelector('.cuns_notice_close_btn');

// 열기
openBtn.onclick = function(e) {
  e.preventDefault();
  modal.style.backgroundColor = 'rgba(0, 0, 0, 0.3)';
  modal.style.display = 'flex';
}

// 닫기
closeBtn.onclick = function() {
  modal.style.display = 'none';
}
```

preventDefault()
생략 가능하지만 a 태그의 링크 이동 방지를 위해 사용

Main Page

메인 페이지




```html
<div class = 'sub_header'>
    <div class = 'sh_main'>
        <!-- sub header logo -->
        <a href = "index.html"><img id = 'sh_logo' src="./img/bold.svg" alt = 'icon'></a>

        <!-- 오른쪽 -->
        <div class = 'icon_pack'>
            <img src="./img/person.png" alt = 'icon'>
            <img src="./img/shopping-bag.png" alt = 'icon'>
            <img src="./img/zoom.png" alt = 'icon'>
        </div>
    </div>
</div>
```

HTML

```js
// sub header

const subHeader = document.querySelector('.sub_header');

window.addEventListener('scroll', function() {

  let scrollTop = window.scrollY;

  if (scrollTop >= 196) {
      subHeader.classList.add('active');
  }

  else {
      subHeader.classList.remove('active');
  }
});
```

JS

```css
.sub_header {
  width: 100%;
  height: 70px;
  display: none;
  position: fixed;
  border-bottom: 1px solid #f0f0f0;
  z-index: 10000;
  background-color: white;
}

.sub_header.active {
  display: block;
  top: 0;
}

.sh_main {
  width: 1170px;
  height: 70px;
  margin: 0 auto;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

CSS

평소에는 감추고 있다가
지정된 픽셀만큼 스크롤을 내리면
서브 헤더가 나오고 화면 상단 고정

```html
<!-- 오른쪽 하단 리모콘 -->
<div class = 'right_remote'>
    <div class = 'remote_group'>
        <img id = 'q_mark' src="./img/question.png" alt = 'icon'>
        <img id = 'to_top' src="./img/top.png" alt = 'icon'>
        <img id = 'kakao' src="./img/kakao.png" alt = 'icon'>
    </div>
</div>
```

HTML

```css
.right_remote {
  width: 63px;
  height: 207px;
  float: right;
  position: fixed;
  right: 20px;
  bottom: 23px;
}
```

CSS

우측 하단에 고정된 리모콘
float로 우측에 고정
bottom으로 하단으로 이동

Sub Page

서브 페이지

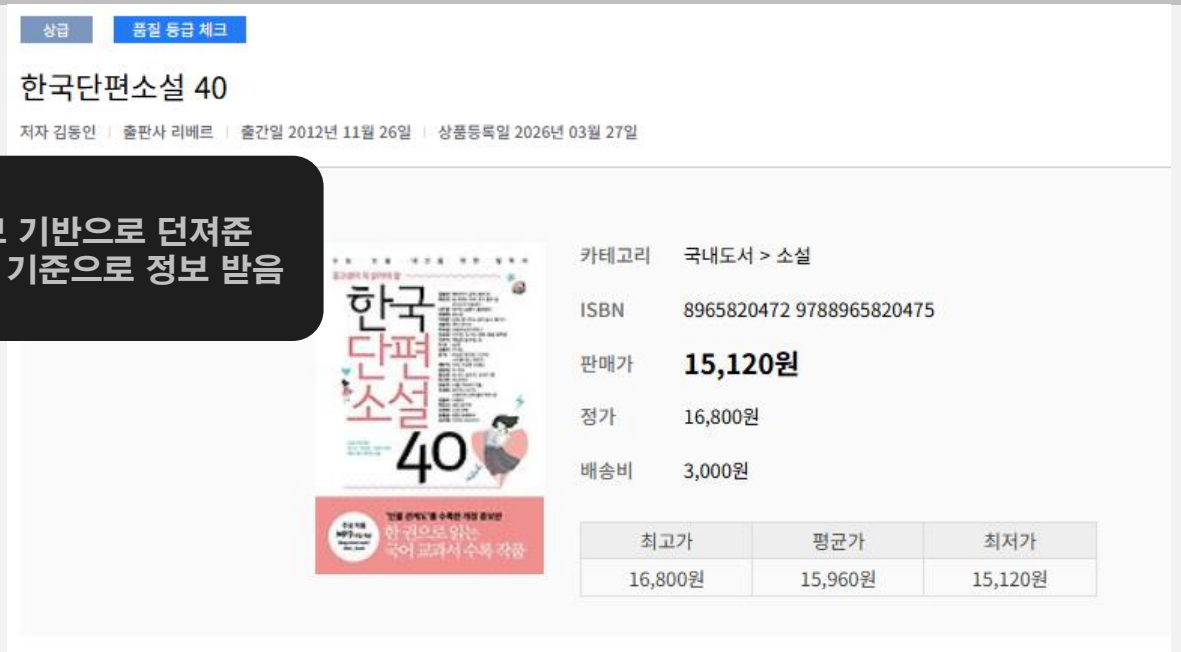
JS - Main

```js
async function loadSubPage(isbn) {
  const response = await fetch('sub/sub.html');
  const htmlText = await response.text();

  document.querySelector('.main_container').innerHTML = htmlText;

  loadData(isbn);
}
```

```js
// 메인에서 넘어온 제목 읽기
window.addEventListener('DOMContentLoaded', () => {
  const urlParams = new URLSearchParams(window.location.search);
  const bookTitle = urlParams.get('title');

  if (bookTitle) {
      loadData(bookTitle);
  }
});
```

JS

메인에서 ISBN 정보 기반으로 던져준
책 정보를 다시 제목을 기준으로 정보 받음



JS

```js
try {
    const response = await fetch(url, {
        method: 'GET',
        headers: {
            Authorization: `KakaoAK ${REST_API_KEY}`
        }
    });

    if (!response.ok) {
        throw new Error(`HTTP 오류 - 상태코드: ${response.status}`);
    }

    const jsonData = await response.json();
    const bookData = jsonData.documents[0];

    if (!bookData) {
        return;
    }

    document.querySelector('.top_title').textContent = bookData.title;

    const getDate = bookData.datetime.substring(0, 10).split('-');
    const setDate = `${getDate[0]}년 ${getDate[1]}월 ${getDate[2]}일`

    const infoList = document.querySelectorAll('.tg_li');
    infoList[0].textContent = `저자 ${bookData.authors.join(', ')}`;
    infoList[1].textContent = `출판사 ${bookData.publisher}`;
    infoList[2].textContent = `출간일 ${setDate}`;
    infoList[3].textContent = "상품등록일 2026년 03월 27일";
```

JS

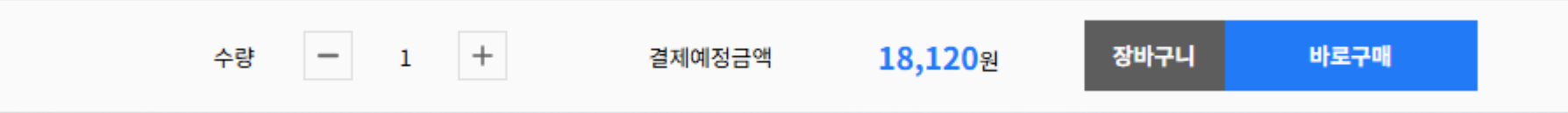
```js
    // 2. 책 이미지
    const bookImg = document.querySelector('.book_info_img');
    bookImg.src = bookData.thumbnail || 'no_image.png'; // 이미지 없으면 기본이미지
    bookImg.alt = bookData.title;

    // 3. 분류 및 가격 상세
    const detailValues = document.querySelectorAll('.bic_value p');
    detailValues[0].textContent = "국내도서 > 소설";
    detailValues[1].textContent = bookData.isbn;
    detailValues[2].textContent = `${bookData.sale_price.toLocaleString()}원`;
    detailValues[3].textContent = `${bookData.price.toLocaleString()}원`;
    detailValues[4].textContent = "3,000원";

    // 4. 테이블 최고가, 평균가, 최저가 출력
    const tablePrices = document.querySelectorAll('.bic_table tr:nth-child(2) td');
    tablePrices[0].textContent = `${bookData.price.toLocaleString()}원`;
    tablePrices[1].textContent = `${((bookData.price + bookData.sale_price)/2).toLocaleString()}원`
    tablePrices[2].textContent = `${bookData.sale_price.toLocaleString()}원`;

    // 5. 하단 고정 테이블 결제 예정 금액
    const bottomPrice = document.querySelectorAll('.bgmc_right p span');
    bottomPrice[0].textContent = `${(bookData.sale_price + 3000).toLocaleString()}`;

} catch (error) {
    console.error(`데이터 로딩 중 에러 발생: ${error}`);
}
}
```

제목 정보를 토대로 받은
책의 정보를 기반으로
서브 페이지에 보여줌

YYYY-MM-DD 형식의 날짜 형식 재정의

Sub Page

서브 페이지



HTML

```html
<!-- 하단 구매 버튼 fixed -->
<div class = 'buy_group'>
    <div class = 'bg_main'>
        <div class = 'bgm_left'>
            <div class = 'bgml_text'>
                <p>수량</p>
            </div>
            <div class = 'bgml_btn_group'>
                <button><img src = '../img/minus.png' alt = 'minus'></button>
                <p>1</p>
                <button><img src = '../img/plus.png' alt = 'plus'></button>
            </div>
        </div>
        <div class = 'bgm_center'>
            <div class = 'bgmc_left'>
                <p>결제예정금액</p>
            </div>
            <div class = 'bgmc_right'>
                <p><span>0</span>원</p>
            </div>
        </div>
        <div class = 'bgm_right'>
            <div class = 'bgmr_bag'>
                <p><a href = '#'>장바구니</a></p>
            </div>
            <div class = 'bgmr_buy'>
                <p><a href = '#'>바로구매</a></p>
            </div>
        </div>
    </div>
</div>
```

CSS

```css
/* 하단 구매 버튼 */
.buy_group {
    width: 100%;
    height: 80px;
    background-color: #FAFAFA;
    z-index: 99999;
    position: fixed;
    bottom: 0;
}
```

서브페이지 하단에 고정되어 있는
구매창은 footer 밑에 작성하고
fixed 및 z-index를 부여해서
계속 고정되어 있는 것처럼 보이게 함

회고 및 리뷰

Problem

- JavaScript를 처음 접하였으나 타 언어와의 유사성 덕분에 이론적 이해는 수월했음. 하지만 실제 단독 구현 시 응용력의 한계를 느꼈음. 이론과 실전의 간극을 줄이기 위해 더 많은 실습과 심화 학습이 필요함을 인지.

- 시간 계획 오차로 계획했던 기능들을 모두 구현하지 못한 점이 아쉬움. 다음 프로젝트는 우선 순위를 지정하며 진행해야겠다고 느낌.

Keep

초기에는 전체적인 레이아웃 위주로 진행하다 보니 세부 구조 설계에 어려움을 겪었음. 이를 통해 단순한 배치 이상의 상세 구조 설계의 중요성을 깨달았으며, 구현 전 기초 설계를 어떻게 수립해야 하는지 명확한 기준을 세울 수 있는 계기가 되었음.

Try

이번 프로젝트에서 느낀 JavaScript 구현의 한계를 보완하기 위해 차기 팀 프로젝트 전까지 핵심 기능 구현 능력을 집중적으로 강화하고, 새롭게 학습하는 내용들까지 부족함이 없게 더욱 정진하겠음

Thank You